LEARN BEING FORWARD

Programming — C Programming

Control flow & functions

COMPLETE LEARNING GUIDE

What You'll Learn

Control flow statements and functions are how a C program moves beyond a single straight-line sequence of instructions — branching on conditions, repeating work, and decomposing a problem into named, reusable, independently testable pieces. C's control-flow constructs (if/else, switch, the three loop forms, break/continue) are the direct ancestors of the equivalent constructs in nearly every language used today, and its function model — pass-by-value parameters, mandatory prototypes, and the ability to take a pointer to a function itself — sets the foundation for everything from simple utility functions to callback-driven event systems. This sub-module works through conditional branching, the three loop constructs and when to choose each, structuring and calling functions correctly, how arrays behave when passed as parameters, recursion, and function pointers.

1. Conditional Branching: if/else and switch

if/else chooses between code paths based on a boolean-like condition (in C, any nonzero value is 'true' and 0 is 'false' — there was no real bool type before C99's stdbool.h). switch compares a single integer or char expression against a list of constant case labels, which is often more readable than a long if-else-if chain when testing one variable against many discrete values, but every case falls through into the next one unless a break statement stops it — intentional fallthrough is occasionally useful (grouping multiple labels for one block of code) but is a very common accidental bug when a break is simply forgotten.

- switch only compares against compile-time constant labels (not variables or ranges) — this is a hard restriction, unlike some other languages' switch/match constructs.
- default (placed anywhere in the switch, though conventionally last) runs when no case matches.

```c
#include <stdio.h>

int main(void) {
    int day = 3;
    switch (day) {
        case 1:
        case 7:
            printf("Weekend\n");
            break;
        case 2:
        case 3:
        case 4:
        case 5:
        case 6:
            printf("Weekday\n");
            break;
        default:
            printf("Invalid day\n");
    }
    return 0;
}
```

2. Loops: for, while, and do-while

for bundles initialization, condition, and increment into one line and is the natural choice when the number of iterations is known or counter-driven. while checks its condition before each iteration and is suited to loops driven by an external condition rather than a counter (reading input until EOF, for instance). do-while checks its condition AFTER executing the body once, guaranteeing at least one execution — useful for menu loops or input-validation loops that must run at least once before anything can be checked. All three support break (exit the loop immediately) and continue (skip to the next iteration's condition check).

- for (int i = 0; i < n; i++) is idiomatic for counting exactly n times; off-by-one errors (<= instead of <) are the most common loop bug.
- do-while's condition is checked at the BOTTOM, so the body always runs at least once, even if the condition is false from the start.

```c
#include <stdio.h>

int main(void) {
    int sum = 0;
    for (int i = 1; i <= 5; i++) {   // runs for i = 1,2,3,4,5
        sum += i;
    }
    printf("sum = %d\n", sum); // 15

    int choice;
    do {
        printf("Enter 1 to continue, 0 to quit: ");
        scanf("%d", &choice);
    } while (choice != 0 && choice != 1); // body runs at least once

    return 0;
}
```

3. break, continue, and the Case Against goto

break immediately exits the nearest enclosing loop or switch; continue skips the rest of the current iteration's body and jumps to the loop's condition check (or increment step, in a for loop). C also has goto, which jumps unconditionally to a labeled statement anywhere in the same function — it predates structured control flow and is almost always avoidable with proper loops and conditionals, with the one widely accepted exception being jumping to a single cleanup/error-handling label at the end of a function to avoid duplicating resource-release code across multiple early-exit points.

- continue in a for loop still runs the increment step before re-checking the condition — it does not skip the increment.
- The accepted goto use case: 'goto cleanup;' to a single exit label that frees resources, avoiding repeated free()/fclose() calls at every early-return point.

4. Function Declaration, Definition, and Prototypes

A function prototype (its return type, name, and parameter types, ending in a semicolon) must appear before a function is called unless the full definition itself appears earlier in the file — without one, older C compilers would implicitly assume a return type of int, which modern C treats as an error. Declaring prototypes in a header file and including that header is the standard way to let multiple source files call each other's functions. A function's return type describes what comes back to the caller; void means nothing is returned, and a function that reaches its closing brace without an explicit return simply returns no value (or garbage, if its return type isn't void).

```c
#include <stdio.h>

int add(int a, int b); // prototype — lets main() call add() before its definition
int main(void) {
    printf("%d\n", add(3, 4)); // 7
    return 0;
}

int add(int a, int b) { // definition
    return a + b;
}
```

5. Parameter Passing: Pass-by-Value and Array Decay

C always passes function arguments by value — the function receives a COPY of each argument, so modifying a parameter inside the function never affects the caller's original variable (this is why 'swap' functions in C must take pointers, covered in the Pointers sub-module). The one apparent exception is arrays: when an array is passed as a function parameter, it silently 'decays' to a pointer to its first element, so the function receives the array's address, not a copy of its contents — this is why sizeof on an array parameter inside a function always returns the size of a pointer, not the original array's size, a very common surprise for beginners.

- void modify(int x) { x = 99; } never changes the caller's variable — x is a local copy.
- void modify(int arr[]) can change the caller's array contents, because arr is really a pointer to the caller's actual data.

```c
#include <stdio.h>

void tryToDouble(int n) {
    n = n * 2; // only changes the local copy
}

void doubleArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // modifies the CALLER's array, because arr decayed to a pointer
    }
}

int main(void) {
    int x = 5;
    tryToDouble(x);
    printf("x = %d\n", x); // 5 — unchanged

    int nums[3] = {1, 2, 3};
    doubleArray(nums, 3);
    printf("%d %d %d\n", nums[0], nums[1], nums[2]); // 2 4 6 — changed
    return 0;
}
```

6. Recursion

A recursive function calls itself with a smaller version of the same problem, always paired with a base case that stops the recursion — without one, the function calls itself indefinitely until it exhausts the call stack and crashes with a stack overflow. Each recursive call gets its own stack frame holding its own copies of local variables and parameters, which is why recursion naturally handles problems with a tree-like or self-similar structure (factorial, Fibonacci, traversing nested data) but can be memory-expensive compared to an equivalent loop for very deep recursion depths.

- Every recursive function needs a base case that returns without recursing further — this is the single most important thing to get right.
- Deep recursion (tens of thousands of calls) risks stack overflow; an iterative loop is usually preferable when depth could be large and unbounded.

```c
#include <stdio.h>

unsigned long long factorial(int n) {
    if (n <= 1) return 1;          // base case — stops the recursion
    return n * factorial(n - 1);   // recursive case
}

int main(void) {
    printf("5! = %llu\n", factorial(5)); // 120
    return 0;
}
```

7. Function Pointers

A function pointer stores the address of a function itself, which lets you pass a function as an argument to another function, store different behaviors in an array, or build simple callback/plugin systems — all without any special language feature beyond pointer syntax. The declaration syntax (return_type (*pointer_name)(parameter_types)) looks intimidating at first but reads the same way as any other pointer: a variable that holds an address, here the address of a function matching that exact signature. This pattern underlies qsort's comparator argument in the C standard library and is the conceptual ancestor of callbacks, delegates, and first-class functions in later languages.

```c
#include <stdio.h>

int add(int a, int b) { return a + b; }
int multiply(int a, int b) { return a * b; }

int applyOperation(int a, int b, int (*op)(int, int)) {
    return op(a, b); // calls whichever function `op` currently points to
}

int main(void) {
    printf("%d\n", applyOperation(3, 4, add));      // 7
    printf("%d\n", applyOperation(3, 4, multiply)); // 12
    return 0;
}
```

Common Pitfalls

- Forgetting a break in a switch statement, causing execution to fall through into the next case unintentionally.
- Off-by-one errors in loop bounds (using <= when < was intended, or vice versa), especially when indexing arrays.
- Expecting a pass-by-value function parameter to modify the caller's original variable, instead of returning the new value or taking a pointer.
- Confusing an array parameter's sizeof (always the size of a pointer) with the original array's full size, after it has decayed inside the function.
- Writing recursion without a correctly-reached base case, or with a base case that never gets hit for certain inputs, causing a stack overflow.
- Calling a function before its prototype or full definition is visible to the compiler, in older toolchains silently assuming an int return type.

Key Takeaways

- switch requires compile-time constant case labels and falls through without an explicit break in each case.
- for suits counter-driven loops, while suits condition-driven loops, do-while guarantees at least one execution.
- C always passes arguments by value; arrays are the apparent exception only because they decay to a pointer, not because C passes them by reference.
- Every recursive function must have a base case reachable from every valid input, or it will eventually overflow the call stack.
- A function prototype must be visible before a function is called from elsewhere in the program.

• Function pointers let you pass behavior as data — the same mechanism behind callbacks in virtually every later language.

Learn More

cppreference — C Control Flow
https://en.cppreference.com/w/c/language/statements

GeeksforGeeks — Functions in C
https://www.geeksforgeeks.org/c/functions-in-c/